

Лекция 5

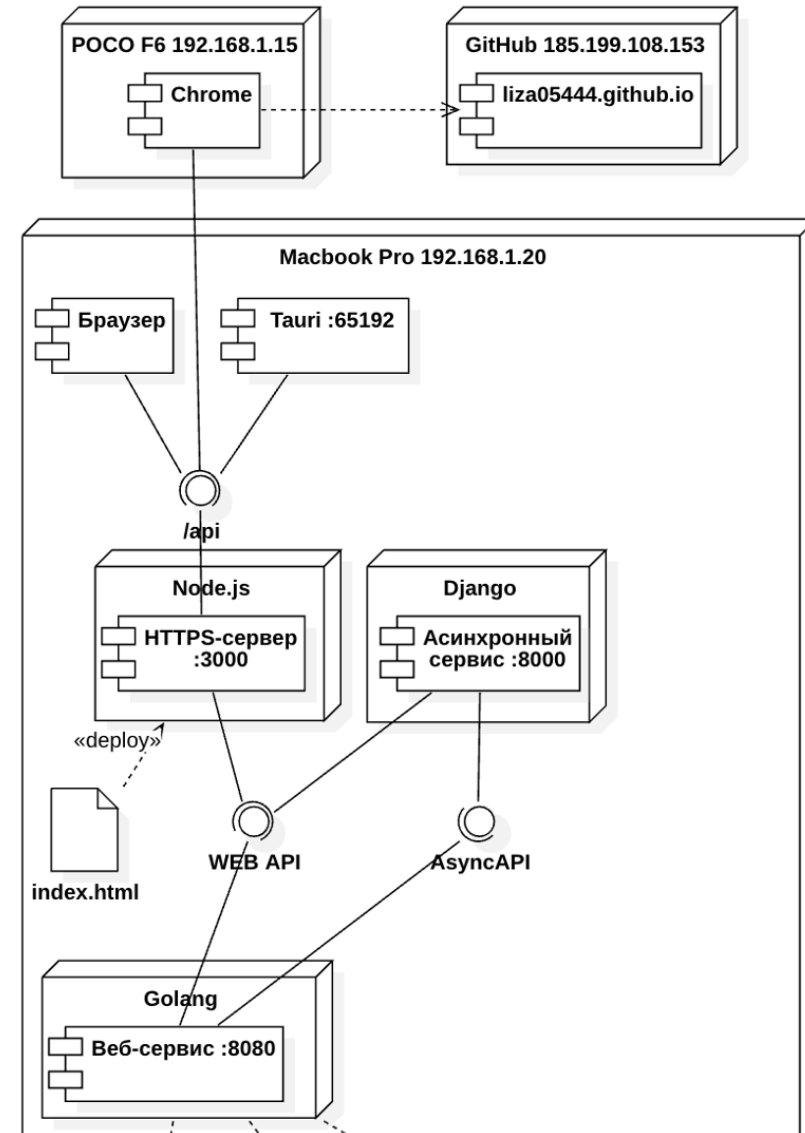
Веб-сервисы, REST, SOA

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Трехзвенная архитектура. API

- Чтобы использовать веб-сервер как универсальный источник данных для других приложений, нам нужно сделать веб-сервис, который предоставит нам Web API
- Теперь данные мы будем получать не как HTML, а структурированно в формате JSON

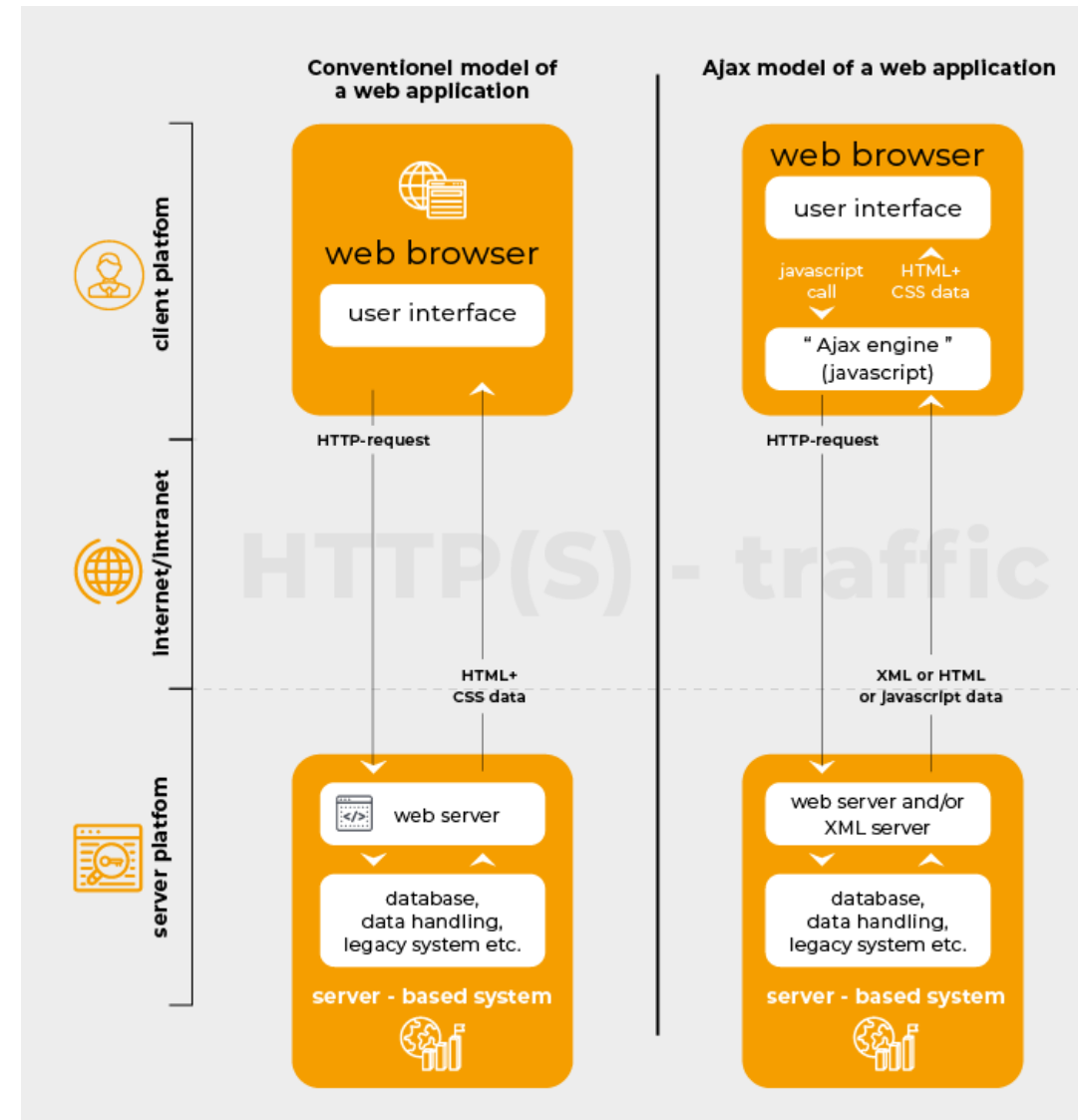


Веб-служба, API *

- **API** (Application Programming Interface) — описание способов взаимодействия одной компьютерной программы с другими. Проще говоря, это набор компонентов (функций), с помощью которых компьютерная программа (бот или же сайт) может использовать другую программу.
- Обычно входит в описание какого-либо интернет-протокола, программного фреймворка или стандарта вызовов функций операционной системы.
- Часто реализуется отдельной программной библиотекой или сервисом операционной системы. **Веб API** реализуется **веб-сервисом**
- Веб-служба, **веб-сервис** (web service) — идентифицируемая уникальным веб-адресом (URL-адресом) программная система со стандартизированными интерфейсами.
- Веб-службы могут взаимодействовать друг с другом и со сторонними приложениями по протоколам (SOAP, JSON-RPC и т. д.) и соглашениях (REST). Веб-служба является единицей модульности для сервис-ориентированной архитектуры приложения.

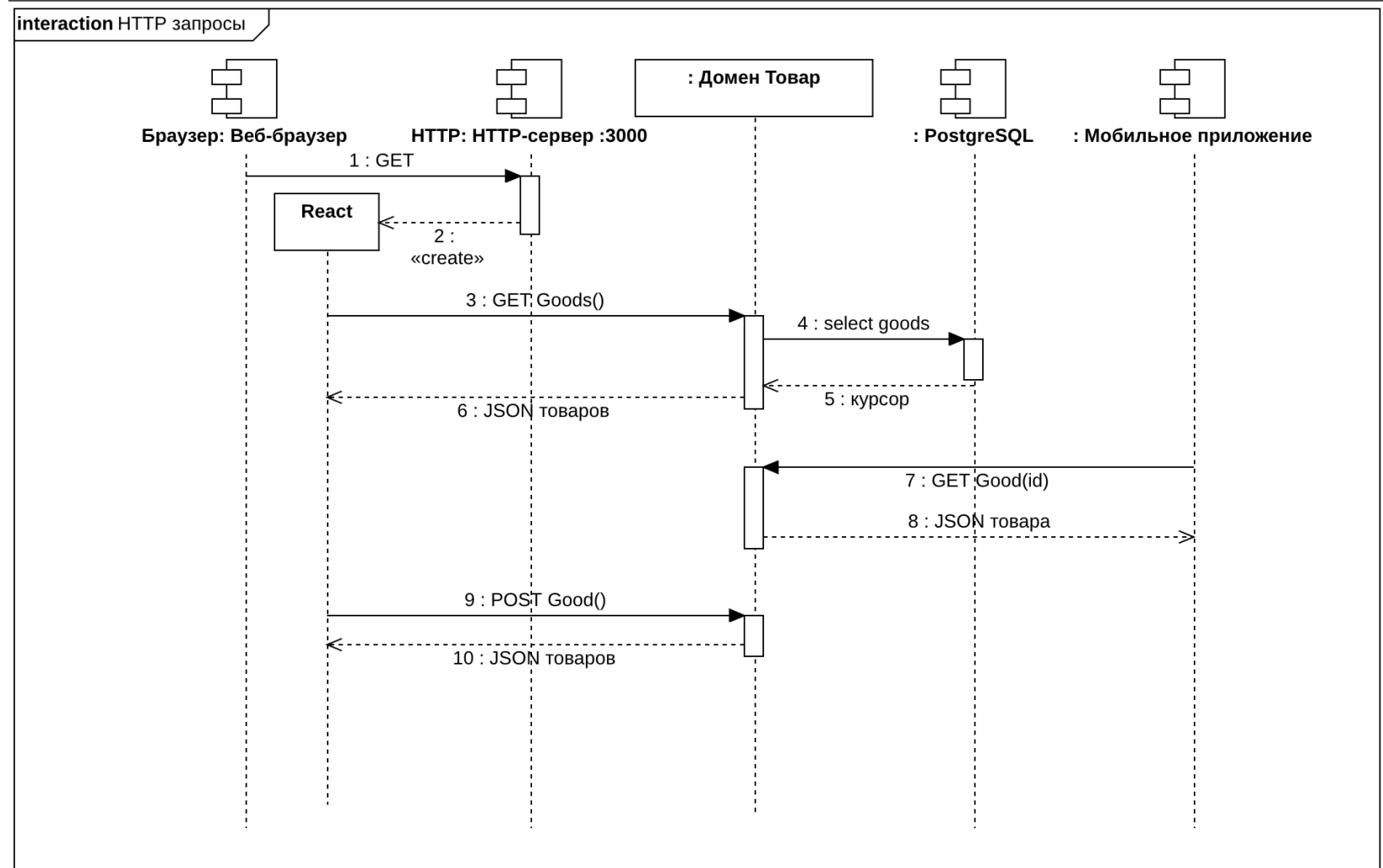
AJAX, JSON-RPC *

- **AJAX**, Ajax (Asynchronous Javascript and XML — «асинхронный JavaScript и XML») — подход к построению интерактивных пользовательских интерфейсов веб-приложений, заключающийся в «фоновом» обмене данными браузера с веб-сервером.
- В результате при обновлении данных веб-страница не перезагружается полностью, и веб-приложения становятся быстрее и удобнее.
- **JSON-RPC** (JavaScript Object Notation Remote Procedure Call — JSON-вызов удалённых процедур) — протокол удалённого вызова процедур, использующий JSON для кодирования сообщений.



AJAX запросы REST API + SPA

- Диаграмму последовательности вы будете делать на 4ой лабораторной
- Мы уже видим два разных сервера: для фронтенда (5-7 лабораторные) и для веб-сервиса
- Мобильное/нативное приложение тоже использует веб-сервис



RPC, gRPC *

- RPC - remote procedure call. Удалённый вызов процедур
- Реализация RPC-технологии включает два компонента: сетевой протокол для обмена в режиме клиент-сервер и язык сериализации объектов (или структур для необъектных RPC)
- Варианты: JSON-RPC, XML RPC (более сложный - SOAP), gRPC
- **gRPC** (Remote Procedure Calls) — это система удалённого вызова процедур (RPC) с открытым исходным кодом, первоначально разработанная в Google (аутентификация, двунаправленная потоковая передача и управление потоком и т.д.)
- В качестве транспорта используется HTTP/2, в качестве языка описания интерфейса — Protocol Buffers.

JSON

- JSON (JavaScript Object Notation) — текстовый формат обмена данными, основанный на JavaScript.
- Заменяет XML как формат данных для HTTP методов

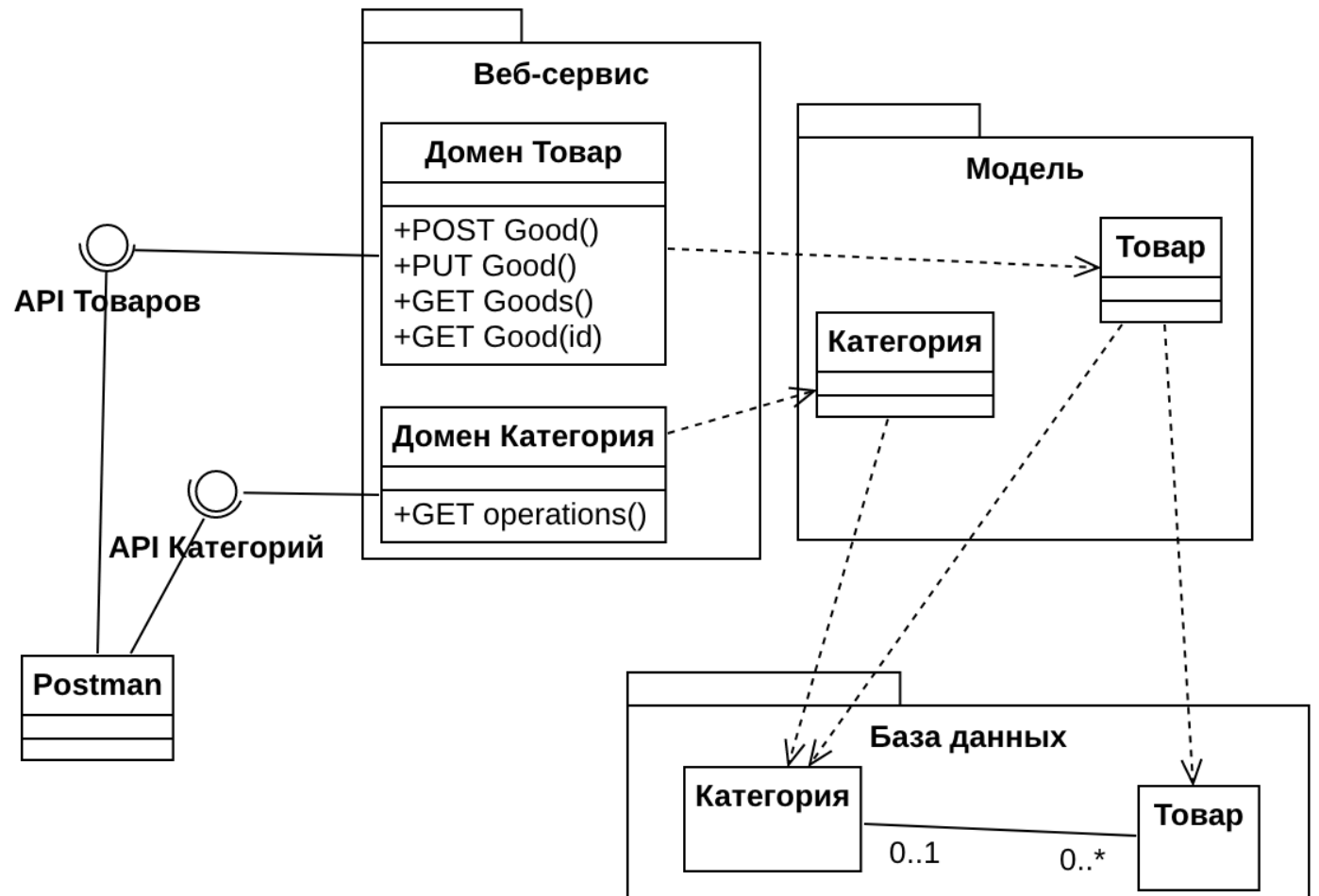
```
<person>
  <firstName>Иван</firstName>
  <lastName>Иванов</lastName>
  <address>
    <streetAddress>Московское ш., 101, кв.101</streetAddress>
    <city>Ленинград</city>
    <postalCode>101101</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>812 123-1234</phoneNumber>
    <phoneNumber>916 123-4567</phoneNumber>
  </phoneNumbers>
</person>
```

```
<person firstName="Иван" lastName="Иванов">
  <address streetAddress="Московское ш., 101, кв.101" city="Ленинград" postalCode="101101" />
  <phoneNumbers>
    <phoneNumber>812 123-1234</phoneNumber>
    <phoneNumber>916 123-4567</phoneNumber>
  </phoneNumbers>
</person>
```

```
{
  "firstName": "Иван",
  "lastName": "Иванов",
  "address": {
    "streetAddress": "Московское ш., 101, кв.101",
    "city": "Ленинград",
    "postalCode": 101101
  },
  "phoneNumbers": [
    "812 123-1234",
    "916 123-4567"
  ]
}
```

Диаграмма классов бэкенда *

- У вас будет **одна диаграмма классов**: 10 методов сервиса, 3 модели + БД, 4 страницы фронтенда
- Нужно добавить страницу авторизации
- Бэкенд предоставляет интерфейс (для хранения данных или вычислений)
- Правильно рисуйте интерфейс, как фонарик, провод входит **ОДИН**, а лучиков света из него **МНОГО**



Методы по вашей теме *

Услуги

- GET список с фильтрацией (только опубликованные). Признак 0/1 если создатель услуги совпадает с текущим пользователем
- GET лента (только опубликованные)
- GET получения черновика (не более 1 записи для пользователя), ид не указывается
- POST добавление услуги + добавление файлов изображения и короткого видео (как файлы а НЕ просто URL)
- PUT публикации (смена статуса) + доп. поля
- DELETE удаление (только soft delete). Только услуги этого пользователя
- POST like (от текущего пользователя). Поле 0/1 (0 отменяет лайк, 1 ставит лайк)

Пользователь

- Регистрация
- Аутентификация (4 лаб)
- Деавторизация (4 лаб)

Модель и сериализация *

```
type Stock struct {  
    ID          uint `gorm:"primaryKey" json:"id"`  
    Name         string `json:"name"`  
    PurchasePrice uint64 `json:"purchase_price"`  
    SalePrice    uint64 `json:"sale_price"`  
    Count        uint64 `json:"count"`  
    CompanyName  string `json:"company_name"`  
    INN          string `json:"inn"`  
    Pic          string `json:"pic"`  
}
```

- **Сериализация** — процесс перевода структуры модели в JSON формат и обратно.
- Каждый раз мы должны получить данные из модели и передать их на клиент в виде JSON. Обратно данные мы получаем также в виде JSON и должны использовать методы модели для изменения БД. Все это делает сериализатор

Обработчики Go*

Для Go необходимо
добавить:

- сериализацию в JSON
- обращение к БД через ORM
- Обработчики HTTP

НЕ добавлять status в тело
ответа, **только коды
состояний** (200/404)

```
func (h *Handler) GetStockByIdAPI(ctx *gin.Context) {
    strId := ctx.Param("id")
    id, err := strconv.Atoi(strId)
    if err != nil {
        h.errorHandler(ctx, http.StatusInternalServerError, err)
        return
    }

    stock, err := h.Repository.GetStock(id)
    if err != nil {
        h.errorHandler(ctx, http.StatusInternalServerError, err)
        return
    }

    ctx.JSON(http.StatusOK, gin.H{
        "status": "success",
        "data":   stock,
    })
}
```

```
router.GET("/stocks", h.GetStocksAPI)
router.GET("/stocks/:id", h.GetStockByIdAPI)
router.POST("/stocks", h.AddStockAPI)
router.PUT("/stocks/:id", h.ModifyStockAPI)
router.DELETE("/stocks/:id", h.DeleteStockAPI)
```

Сериализация вложенность

- Вложенные модели нужно передавать в том же запросе
- Поэтому сериализатор должен поддерживать вложенность
- А лучше передать сериализатор в качестве декларируемого поля

```
class TownSerializer(serializers.ModelSerializer):
    class Meta:
        model = Town
        fields = ['id', 'name', 'writers']

s = TownSerializer(instance=Town.objects.first())
print(s.data)
-----
{'id': 1, 'name': 'Вологда', 'writers': [6, 7]}
```

```
class TownSerializer(serializers.ModelSerializer):
    class Meta:
        model = Town
        depth = 1
        fields = ['id', 'name', 'writers']

s = TownSerializer(instance=Town.objects.first())
print(s.data)
-----
{
    'id': 1,
    'name': 'Вологда',
    'writers': [
        {'id': 6, 'firstname': 'Варлам', 'lastname': 'Шаламов', ...},
        {'id': 7, 'firstname': 'Константин', 'lastname': 'Батюшков', ...}
    ]
}
```

```
class TownSerializer(serializers.ModelSerializer):
    writers = WriterSerializer(many=True) # Добавили сериалайзер в качестве поля

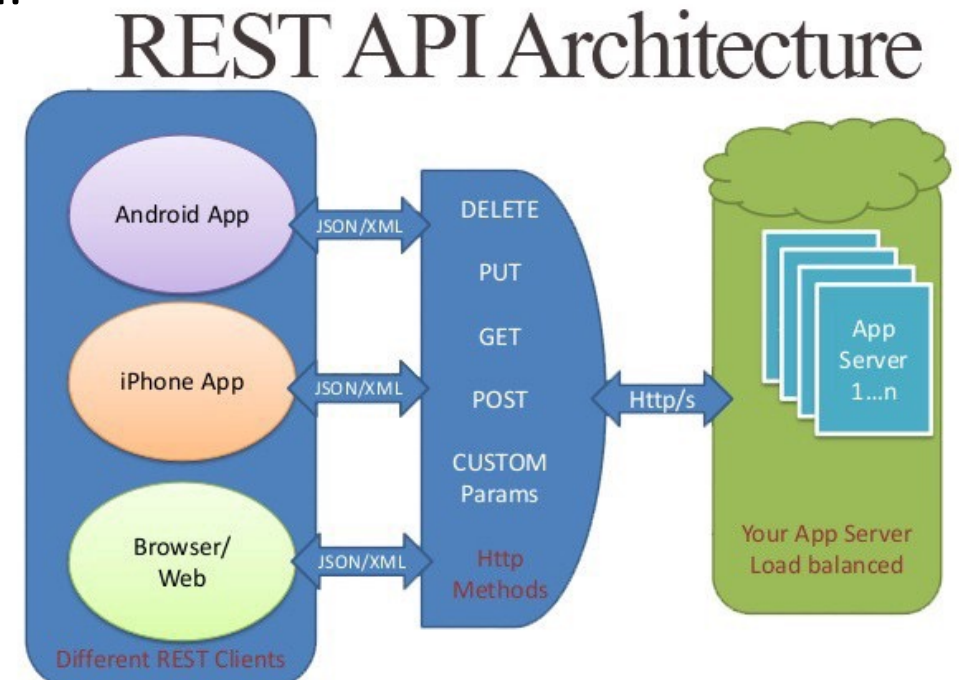
    class Meta:
        model = Town
        fields = ['id', 'name', 'writers']
```

REST – формальные требования к HTTP *

- REST (Representational State Transfer) – набор правил того, как программисту организовать написание кода серверного приложения, чтобы все системы легко обменивались данными и приложение можно было масштабировать.
- REST предложил Рой Филдинг для HTTP (до API)

RESTful – веб-службы не нарушающие **ограничений**:

- Клиент-сервер (дорога нас не спрашивает)
- Отсутствие состояния (не важно откуда едем)
- Кэширование (сохранить ответ на клиенте - вы можете запомнить дорогу)
- Единообразие интерфейса (CAMOE сложное, правильно оформить методы и адреса)
- Слои (проксирование/балансировка, смотрим на дорогу/знаки/разметку)
- Код по требованию (JS в браузере, как навигатор)



URL API, Правила именования REST *

<https://habr.com/ru/articles/351890/>

1. Конечные точки в URL – имя существительное, не глагол

2. Множественное число

3. Документация

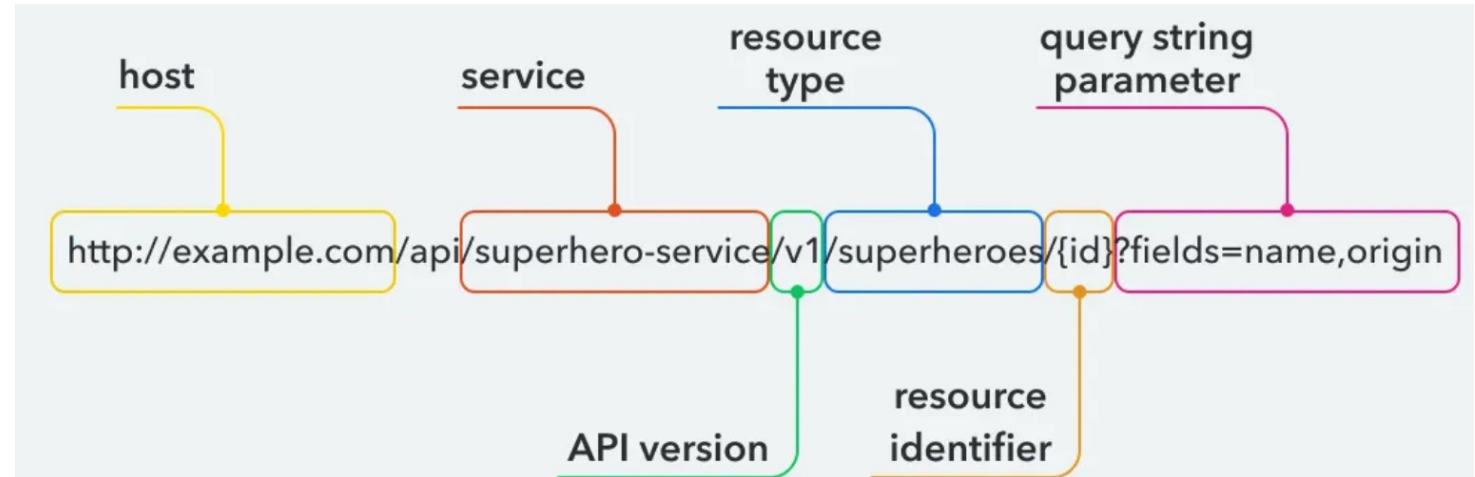
4. Версия вашего приложения

5. Пагинация

6. Использование SSL

7. HTTP методы

8. Эффективное использование кодов ответов HTTP



- В url указываем домен (группа методов), название метода и идентификаторы
- Тут не лучший пример
- Вам нужно **написать** «/api»

```
urlpatterns = [  
    path('', include(router.urls)),  
    path(r'stocks/', views.get_list, name='stocks-list'),  
    path(r'stocks/post/', views.post_list, name='stocks-post'),  
    path(r'stocks/<int:pk>/', views.get_detail, name='stocks-detail'),  
    path(r'stocks/<int:pk>/put/', views.put_detail, name='stocks-put'),  
    path(r'stocks/<int:pk>/delete/', views.delete_detail, name='stocks-delete'),  
    path('api-auth/', include('rest_framework.urls', namespace='rest_framework')),  
  
    path('admin/', admin.site.urls),  
]
```

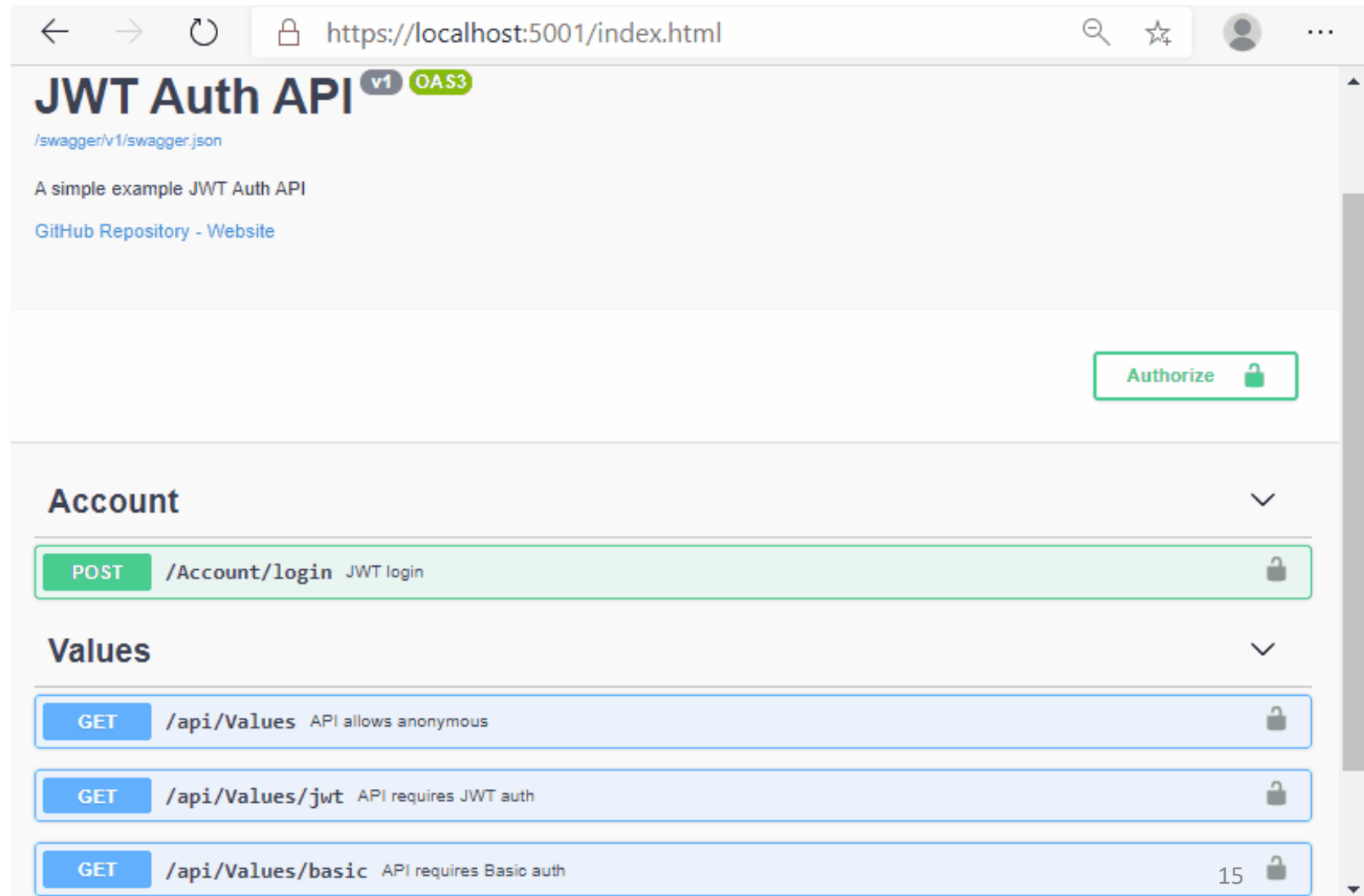
Инструменты тестирования

Инструменты для
нашего курса

- Postman
- Insomnia

Очень удобно, но
необходимо
реализовать

- Swagger
- Swagger UI



Swagger *

Разработка API (4 лабораторная)

- При создании API может использоваться Swagger Tooling для **автоматической генерации** открытого описания API на основе вашего кода. Это встраивает описание API в исходный код и называется code-first или разработкой API снизу вверх.
- В качестве альтернативы, используя Swagger Codegen, разработчики могут **сначала согласовать документ**, описание методов Open API и после написать код клиента и сервера непосредственно из проектирования

Взаимодействие с API (7 лабораторная)

- Используя проект **Swagger Codegen**, конечные пользователи генерируют клиентские SDK непосредственно из документа OpenAPI, сокращая объем кода программиста

Документирование API (4 лабораторная, альтернатива Postman)

- При описании документа OpenAPI, Swagger с открытым исходным кодом может использоваться для непосредственного взаимодействия с API через пользовательский интерфейс **Swagger UI**.

Insomnia

The screenshot displays the Insomnia REST client interface. The top bar shows the title "BMSTU Lab 6 – create new stock". The main area is divided into three panels. The left panel shows the request details: a POST request to "http://127.0.0.1:8000/stocks/" with a status of "201 Created", a response time of "8.34 ms", and a size of "110 B". The request body is a JSON object: `{"company_name": "Mail.Ru", "price": 100500, "is_growing": true}`. The middle panel shows the request headers, query parameters, and docs. The right panel shows the response body, which is a JSON array of three objects, each representing a stock entry with fields like "pk", "company_name", "price", "is_growing", and "date_modified".

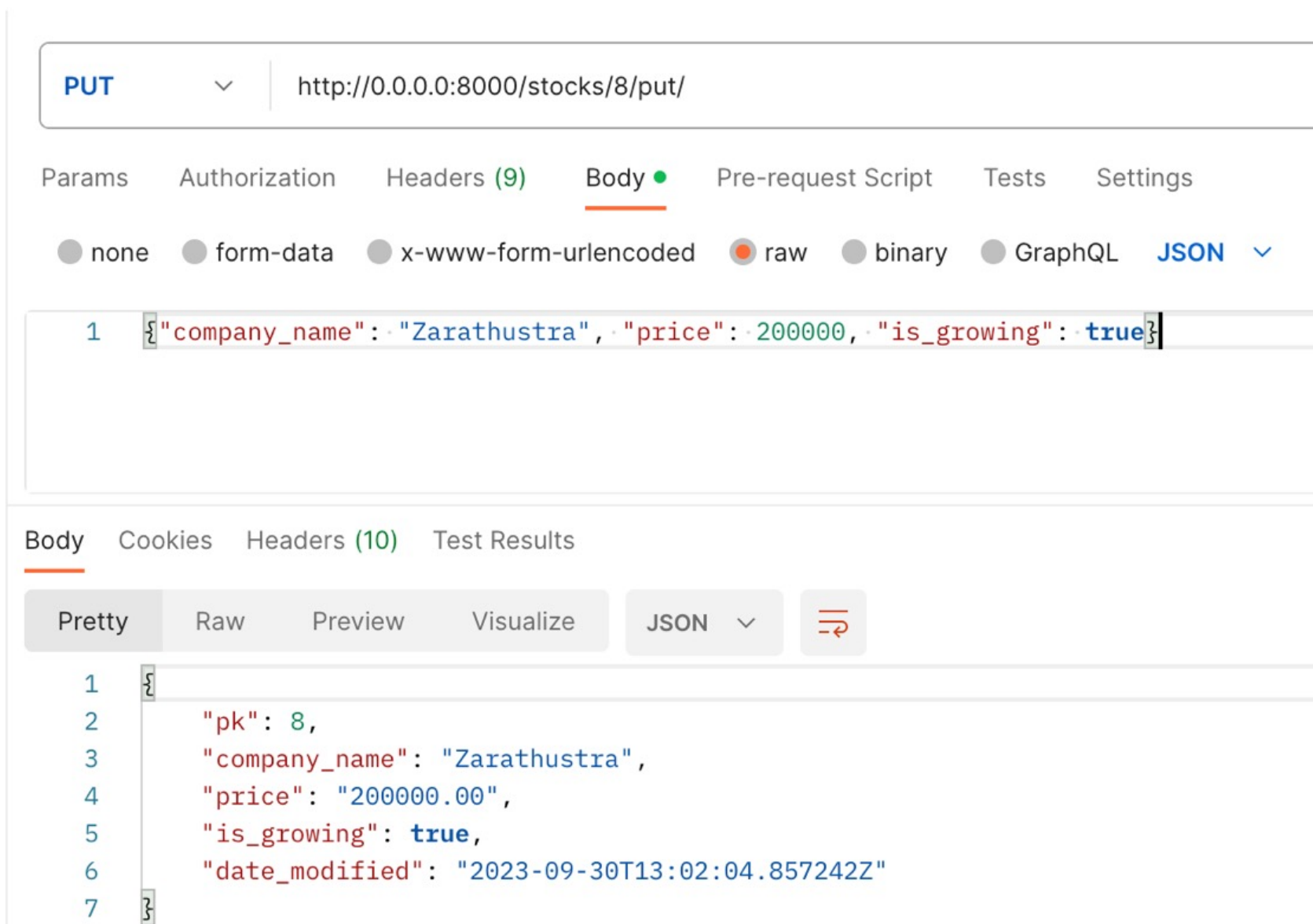
Выбор типа тела запроса JSON

JSON структура запроса

JSON структур ответа

Postman

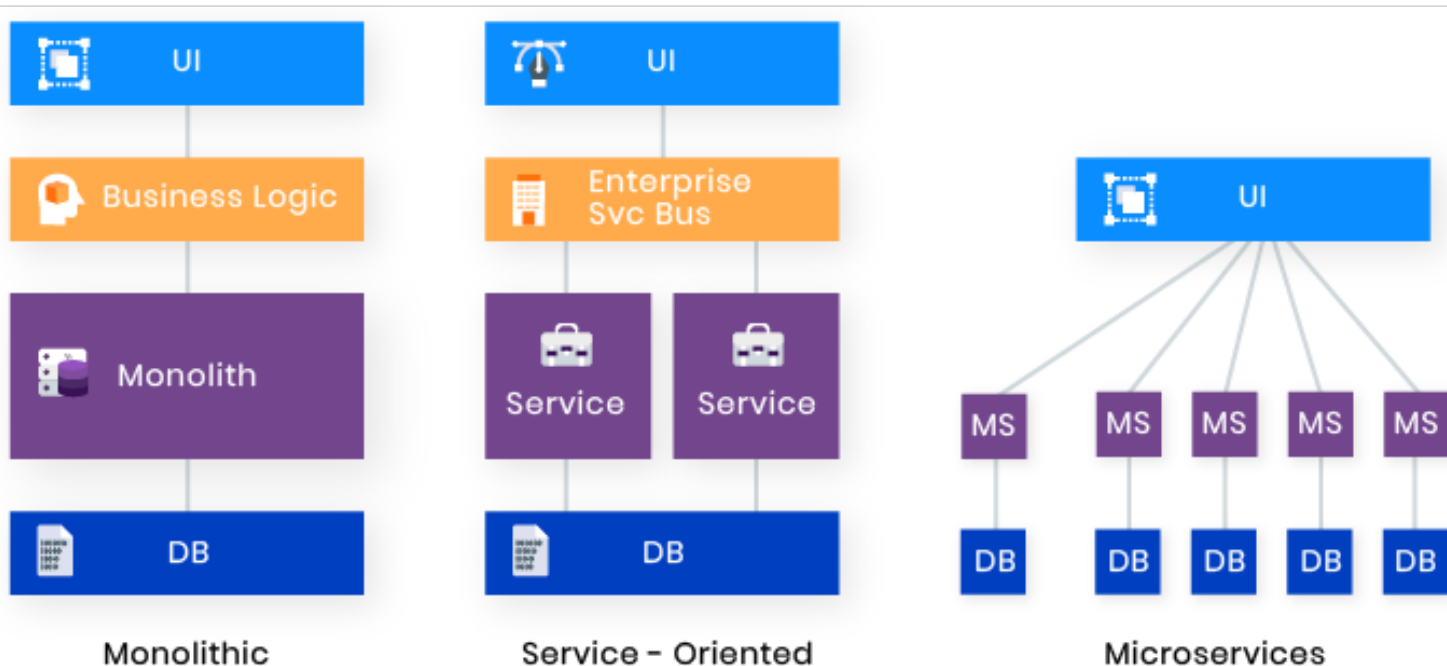
- Другое распространённое приложение для тестирования HTTP-методов
- Также можно тестировать gRPC-методы



Сервис-ориентированная архитектура *

- **Сéрвис-ориентірованная архитектура** (COA, англ. service-oriented architecture- SOA) — модульный подход к разработке ПО на основе использования по стандартизированным протоколам распределённых, слабо связанных и легко заменяемых сервисов со стандартизированными интерфейсами.

- **Микросервисная архитектура** — вариант сервис-ориентированной архитектуры программного обеспечения на основе взаимодействия небольших, слабо связанных и легко изменяемых модулей - микросервисов



- Теперь каждая система состоит из множества отдельных сервисов, выполняющих самостоятельную бизнес-функцию
- Например можно разделить услуги, заявки и авторизацию
- Получила распространение в середине 2010-х годов в связи с развитием практик гибкой разработки и DevOps

S3 *

- S3 - Simple Storage Service
- Сeph, Minio и тд

Создание озера данных

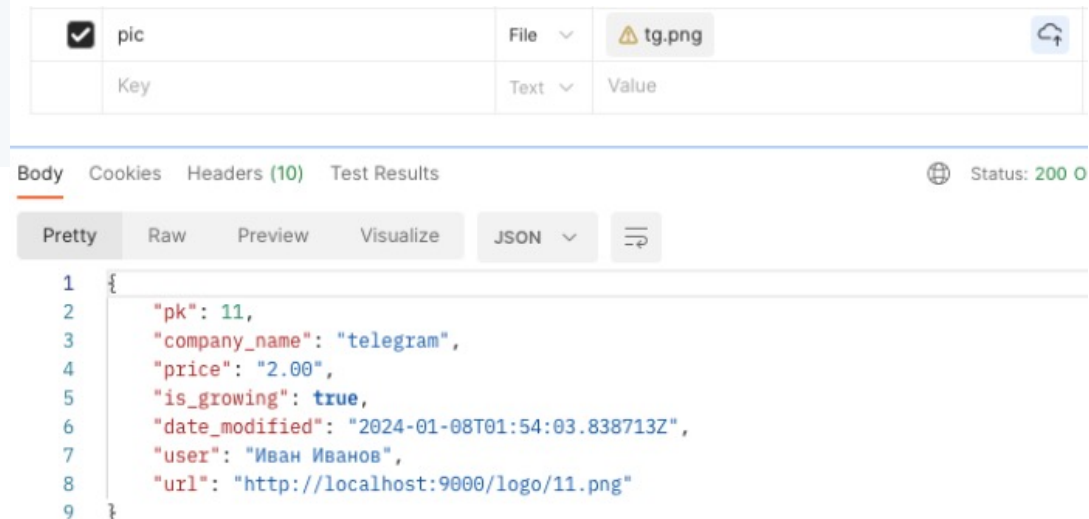
- Приложения для аналитики больших данных, искусственного интеллекта (ИИ), машинного обучения (ML) и высокопроизводительных вычислений (HPC).
- Объектное хранилище S3 — масштабируемый и гибкий сервис, позволяющий хранить и обрабатывать данные в исходном формате
- Данные размещаются в виде объектов в плоском адресном пространстве, что дает возможность для доступа к хранилищу по API из любого места
- Данные помещаются во множество контейнеров (папок)
- Содержимое любого контейнера можно просматривать, перемещать или удалять
- У каждого контейнера и объекта есть адрес в виде уникального идентификатора

Minio загрузка изображения *

- Мы не загружаем изображения напрямую в Minio из браузера, потому что далее мы ограничим на это права пользователей
- Нам требуется загрузить/заменить изображение в Minio, а в Postgres - url

```
rep, errRep := repository.New(&repository.RepositorySettings{
    PostgresDSN: postgresString,
    MinioEndpoint: os.Getenv("MINIO_ENDPOINT"),
    MinioAccessKey: os.Getenv("MINIO_ACCESS_KEY"),
    MinioSecretKey: os.Getenv("MINIO_SECRET_KEY"),
    MinioBucketName: os.Getenv("MINIO_BUCKET_NAME"),
```

```
_, err = r.minio.PutObject(
    r.minio_bucket_name,
    filename,
    file,
    header.Size,
    minio.PutObjectOptions{
        ContentType: contentType,
    })
```

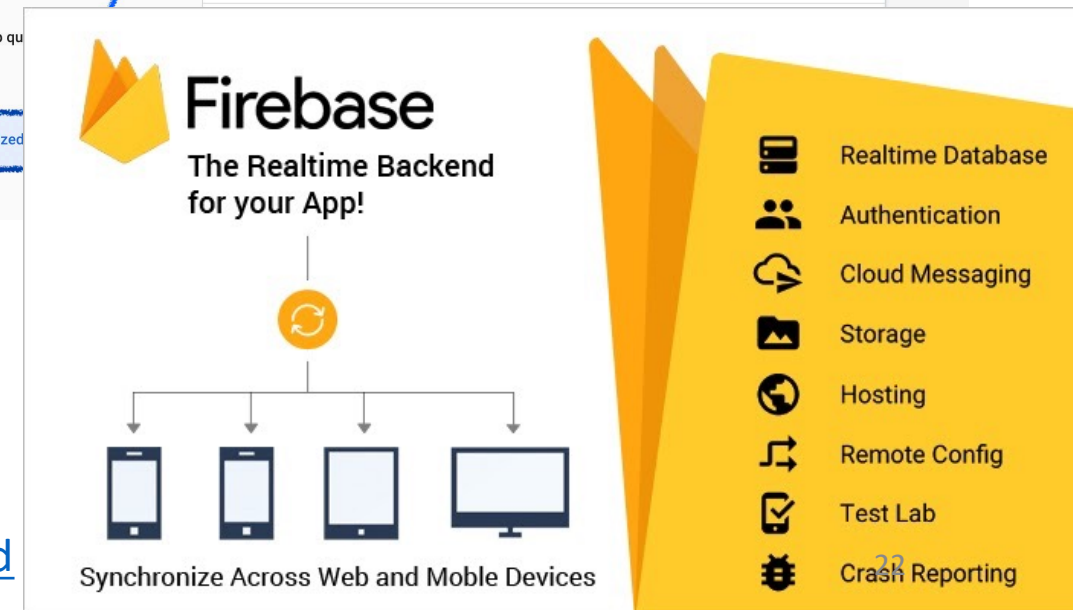
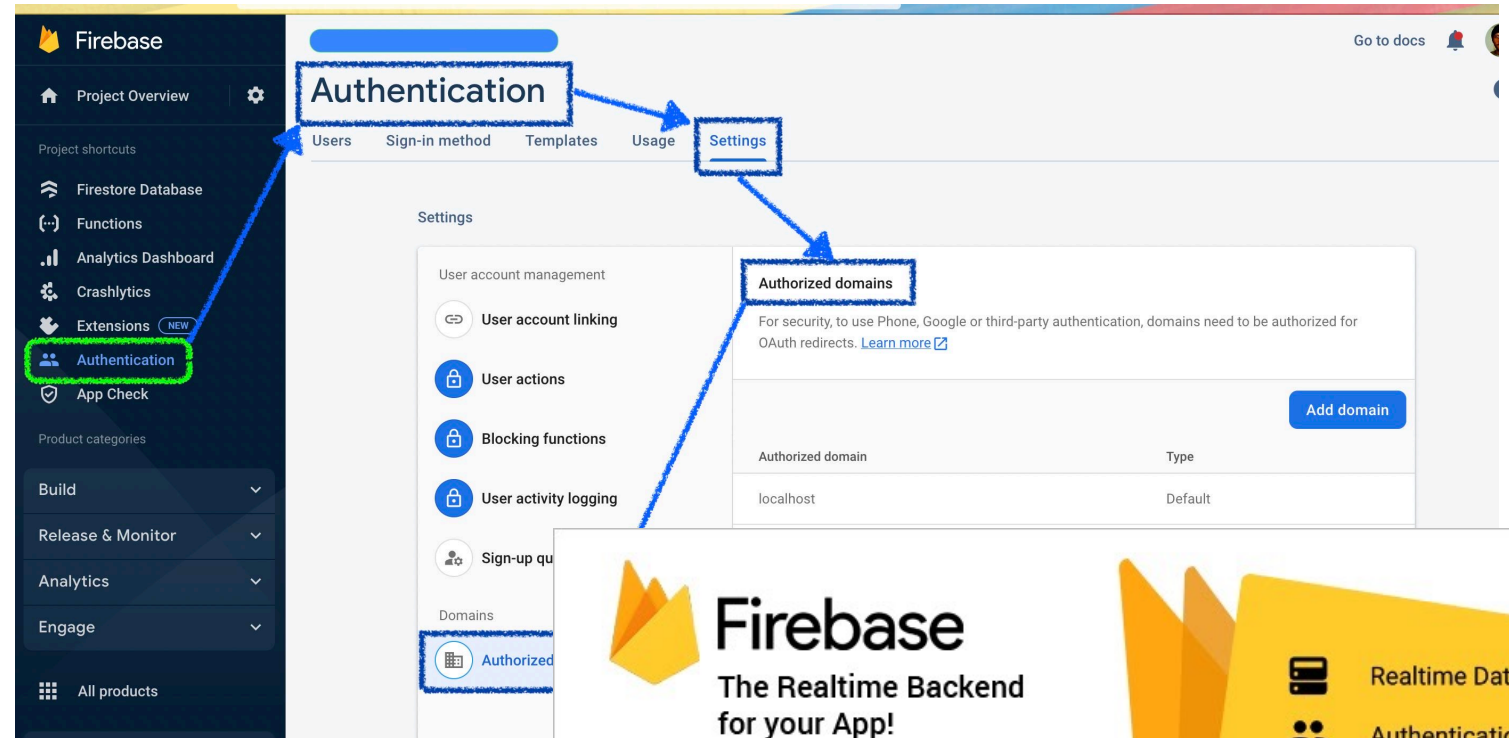


Облачные сервисы и Firebase

- Хостинги для вашего бекенда: Heroku, AWS и тд
- Платно + администрирование
- Любая архитектура

Firebase

- Удобно использовать для мобильных приложений
- Low/No code – готовая конфигурация



<https://github.com/iu5git/Web/blob/main/tutorials/firebase/README.md>